INITIALISATION

`initCatDist(nvars, ncats, layers; shift=0.1)`

Allocate weights & biases arrays for a categorical distribution network. Output layer biases set to 0; hidden layers atanh-spaced + shift.

→ (w, b, npars) :: (Array, Array, Int)

nvars	Int	inputs including the category column
ncats	Int	number of categories
layers	Vector{Int}	hidden layer widths
shift KW	Float64	bias init offset

`initConDist(nvars, layers1, layers2; shift=0.1)`

Allocate weights & biases arrays for a continuous distribution network. Output layer bias set to 0; hidden layers atanh-spaced + shift.

→ (w, b, npars) :: (Array, Array, Int)

nvars	Int	total inputs including target
layers1	Vector{Int}	pre-warp hidden layer widths
layers2	Vector{Int}	post-warp hidden layer widths
shift KW	Float64	bias init offset

`initTimDist(nvars, nevents, layers1, layers2; shift=0.1)`

Allocate weights & biases arrays for a time-to-event with competing-risks network. Event-prob sub-layer and output biases set to 0; others atanh-spaced + shift.

→ (w, b, npars) :: (Array, Array, Int)

nvars	Int	inputs incl. time-start & time-end columns
nevents	Int	number of competing events
layers1	Vector{Int}	pre-warp layer widths
layers2	Vector{Int}	post-warp layer widths
shift KW	Float64	bias init offset

TRAINING

`fitCatDist(w, b, data, categories, layers, iterations; …)`

Train categorical distribution via Adam on log-likelihood. Sequential sigmoid encoding. Mutates weights & biases.

→ (w, b, llv) — updated params + log-likelihood trace Vector{Float64}

w, b	Array	weights & biases (mutated)
data	Matrix	N × (nvars[+1]) observations
categories	Vector	ordered category labels
layers	Vector{Int}	hidden layer widths
iterations	Int	Adam steps
hotvars KW	Vector{Int}	sigma-hot group sizes
obsweights KW	Bool	last data col = obs weight
α β1 β2 ϵ KW	Float64	Adam hyperparameters
c KW	Vector{Float64}	weight clamp [c[1]=input, c[2]=rest]

`fitConDist(w, b, data, layers1, layers2, iterations; …)`

Train continuous distribution via Adam on log-likelihood. Jacobian via warp layer. Mutates weights & biases.

→ (w, b, llv) — updated params + log-likelihood trace Vector{Float64}

w, b	Array	weights & biases (mutated)
data	Matrix	N × (nvars[+1]) observations
layers1/2	Vector{Int}	pre-/post-warp layer widths
iterations	Int	Adam steps
hotvars KW	Vector{Int}	sigma-hot group sizes
obsweights KW	Bool	last data col = obs weight
α β1 β2 ϵ KW	Float64	Adam hyperparameters
c KW	Vector{Float64}	weight clamp [c[1]=input, c[2]=rest]

`fitTimDist(w, b, data, events, layers1, layers2, iterations; …)`

Train time-to-event network via Adam on log-likelihood. Handles exact events and interval-censored observations via two forward passes. Mutates weights & biases.

→ (w, b, llv) — updated params + log-likelihood trace Vector{Float64}

w, b	Array	weights & biases (mutated)
data	Matrix	N × (nvars+2[+1]); cols nvars & nvars+1 = time interval
events	Vector	event labels, length = nevents
layers1/2	Vector{Int}	pre-/post-warp layer widths
iterations	Int	full Adam steps
hotvars KW	Vector{Int}	sigma-hot group sizes
obsweights KW	Bool	last data col = obs weight
α β1 β2 ϵ KW	Float64	Adam hyperparameters
c KW	Vector{Float64}	weight clamp [c[1]=input, c[2]=rest]

EVALUATION — PDF

`evalCatPDF(obs, ncats, layers, w, b)` Returns the probability vector over all categories. Walks sequential sigmoid chain; last category gets remaining mass. → pcats :: Vector{Float64} (length = ncats, sums to 1)	`obs`	`Vector{Float64}`	covariates; last element = category value
	`ncats`	`Int`	number of categories
	`layers, w, b`		trained network

`evalConPDF(obs, layers1, layers2, w, b)` Continuous density at a single observation. Returns Jacobian from the forward pass. → Float64 — density value	`obs`	`Vector{Float64}`	covariates + target value (last element)
	`layers1/2, w, b`		trained network

`evalTimPDF(obs, nevents, layers1, layers2, w, b)` Joint event-probability vector and per-event conditional density vector for all competing events. → (pevents, devents) :: (Vector{Float64}, Vector{Float64}) each length = nevents	`obs`	`Vector{Float64}`	covariates; obs[end-1]=event label, obs[end]=time
	`nevents`	`Int`	number of competing events
	`layers1/2, w, b`		trained network

EVALUATION — CDF

`evalConCDF(obs, layers1, layers2, w, b)` CDF for continuous distribution at a single observation. Sigmoid at output layer; no Jacobian needed. → Float64 ∈ (0, 1)	`obs`	`Vector{Float64}`	covariates + target value
	`layers1/2, w, b`		trained network

`evalTimCDF(obs, nevents, layers1, layers2, w, b)` Event-probability vector and per-event conditional time CDF vector at a single observation. → (pevents, devents) :: (Vector{Float64}, Vector{Float64}) each length = nevents	`obs`	`Vector{Float64}`	covariates; obs[end-1]=event label, obs[end]=time
	`nevents`	`Int`	number of competing events
	`layers1/2, w, b`		trained network

CDF LIMITS

`limitsConCDF(layers1, layers2, w, b)` Infimum and supremum of the continuous CDF, used for scaling when computing quantiles. → (inf, sup) :: (Float64, Float64)	`layers1/2, w, b`		trained network (no observation needed)

`limitsTimCDF(nevents, layers1, layers2, w, b)` Per-event infimum and supremum of the time CDF, used for scaling when computing quantiles. → (inf, sup) :: (Vector{Float64}, Vector{Float64}) each length = nevents	`nevents`	`Int`	number of competing events
	`layers1/2, w, b`		trained network (no observation needed)

QUANTILES

`quantileCon(cvars, p, layers1, layers2, w, b; accuracy, boundLow, boundUpp)` Bisection-search quantile for continuous distribution. Bracket expands geometrically until p is enclosed. → Float64 — target value at cumulative probability p	`cvars`	`Vector{Float64}`	covariates (no target)
	`p`	`Float64`	probability level ∈ (0,1)
	`layers1/2, w, b`		trained network
	`accuracy` KW	`Float64`	bisection convergence tolerance
	`boundLow/Upp` KW	`Float64`	maximum bracket expansion limits

`quantileTim(cvars, eventnr, nevents, p, layers1, layers2, w, b; accuracy, boundLow, boundUpp)` Bisection-search quantile for time distribution of a specific competing event. Bracket expands geometrically until p is enclosed. → Float64 — time at cumulative probability p for event eventnr	`cvars`	`Vector{Float64}`	covariates (no time)
	`eventnr`	`Int`	event index (1..nevents) to query
	`nevents`	`Int`	number of competing events
	`p`	`Float64`	probability level ∈ (0,1)
	`layers1/2, w, b`		trained network
	`accuracy` KW	`Float64`	bisection convergence tolerance
	`boundLow/Upp` KW	`Float64`	maximum bracket expansion limits